

Explicación detallada de RankGA

Documento técnico alineado con la implementación actual del repositorio

7 de julio de 2026

1. Propósito de este documento

Este documento explica con detalle el algoritmo implementado en el repositorio **RankGA**. No describe un algoritmo genérico inventado aparte del código, sino la versión concreta que hoy existe en las clases `rankga.RankGA`, `rankga.Population`, `rankga.Individual`, `rankga.Problem` y `rankga.Gene`.

La meta es dejar claro:

- qué problema resuelve el framework;
- cuál es la arquitectura conceptual del sistema;
- cómo opera cada fase de la evolución;
- qué ecuaciones usa realmente la implementación;
- qué información genera como salida;
- y cuáles son sus supuestos, limitaciones y detalles que pueden provocar mala interpretación si solo se mira el nombre de las clases.

Este texto es deliberadamente exhaustivo. La idea no es solo decir “qué hace”, sino dejar explícito *por qué* está organizado así, qué convenciones adopta y dónde conviene tener cuidado.

2. Idea general del algoritmo

RankGA es un algoritmo genético con cuatro rasgos estructurales centrales:

1. **Selección por rango.** La probabilidad esperada de supervivencia o replicación de un individuo no depende directamente de su valor numérico de fitness, sino de su *posición ordenada* dentro de la población.
2. **Mutación dependiente del rango.** La intensidad de mutación no es uniforme en toda la población. Los individuos ubicados al inicio de la población ordenada por fitness reciben menos perturbación y los ubicados al final reciben más.
3. **Emparejamiento por rango para recombinación.** La recombinación no empareja individuos al azar. Después de ordenar la población por fitness, el algoritmo empareja individuos contiguos en ese orden. Por tanto, la estructura del rango no solo controla la selección y la mutación, sino también *quién se recombina con quién*.
4. **Separación entre motor y problema.** El motor evolutivo es genérico y delega en el problema concreto la forma de representar genes, construir individuos, evaluar fitness y definir el significado de la intensidad de mutación.

La orientación del framework es de **maximización**. Un fitness mayor siempre es mejor. Toda la lógica de ordenamiento y selección está escrita con esa convención.

3. Arquitectura del sistema

3.1. Componentes principales

El núcleo del framework se reparte en estas abstracciones:

Componente	Responsabilidad principal
Problem	Define el problema de optimización: fitness, longitud de genoma, construcción de genes e individuos, intensidad local, intensidad global y fitness objetivo.
Gene	Define la unidad atómica del genoma. Cada implementación decide cómo almacenar el valor, cómo mutarlo y cómo medir distancia a otro gen homólogo.
Individual	Representa una solución completa: genoma, fitness, rango, intensidad de mutación recibida y operaciones de mutación y recombinación a nivel del individuo.
Population	Encapsula las operaciones poblacionales: evaluación, ordenamiento por fitness, selección, recombinación y mutación.
RankGA	Es el <i>driver</i> de la corrida: inicializa, itera, registra salidas, decide el paro y genera resúmenes y gráficas.
AdaptiveProblem	Interfaz opcional para problemas cuyo estado interno puede adaptarse durante la corrida.
ProblemFactory	Selección explícita del problema desde la línea de comandos.

3.2. Separación entre motor y codificación

El framework no impone una sola codificación:

- puede haber genes enteros categóricos;
- genes reales de precisión doble;
- y cualquier otra representación que respete la interfaz **Gene**.

Esto tiene una consecuencia importante: la palabra *mutación* no significa lo mismo en todas las implementaciones. El motor solo calcula una **intensidad** y la pasa a cada gen. El **Gene** concreto decide qué hacer con esa intensidad.

4. Contrato formal entre problema y motor

4.1. La interfaz Problem

Todo problema debe proveer:

- `fitness(Individual)`: evalúa la calidad de una solución;
- `getProblemName()`: nombre visible del problema;
- `getGenomeLength()`: longitud del genoma;

- `getNewGene(boolean, Random)`: construcción de genes nuevos;
- `getNewGene(Gene)`: copia o proyección de un gen existente;
- `getGoalFt()`: fitness objetivo que, si se alcanza, detiene la corrida;
- `getDisplayModulus()`: agrupación visual del genoma en logs;
- `getNewIndividual(boolean, Random)`: construcción de individuos nuevos;
- `getNewIndividual(Individual)`: copia profunda de un individuo;
- `getLocalSearchIntensity()` y `getGlobalSearchIntensity()`.

4.2. Interpretación de las intensidades L y G

El problema define dos parámetros:

- L : intensidad local;
- G : intensidad global.

En el framework actual:

- G es la intensidad máxima asignable y se aplica al individuo de peor fitness, que queda al final de la población ordenada y tiene rango normalizado $r = 1$;
- L se usa para calibrar la pendiente del perfil de mutación.

G debe interpretarse como una intensidad de exploración global. Debe elegirse de modo que una mutación con intensidad G pueda llevar al individuo a cualquier región del espacio de búsqueda. Idealmente eso debe ser posible en una sola generación. En aplicaciones prácticas puede aceptarse una calibración menos agresiva si permite cubrir cualquier región en dos o tres generaciones, pero la idea conceptual sigue siendo la misma: G no es sólo una mutación “grande”, sino una escala de mutación capaz de romper la vecindad local del individuo actual.

Para genes discretos con `numValues` valores posibles por posición, una calibración natural de la intensidad global es

$$G = 1 - \frac{1}{\text{numValues}}.$$

La razón es que, si un valor se remuestrea uniformemente entre `numValues` posibilidades, la probabilidad de que el valor resultante sea distinto del valor actual es $1 - 1/\text{numValues}$. En `GeneInteger`, la mutación se implementa de forma equivalente para esta calibración: con probabilidad G se elige uniformemente un valor distinto del actual, y con probabilidad $1 - G$ se conserva el valor actual. Si $G = 1 - 1/\text{numValues}$, el valor actual y cada valor alternativo quedan con probabilidad final $1/\text{numValues}$. El caso binario es el caso particular `numValues` = 2, por lo que

$$G = 1 - \frac{1}{2} = 0,5.$$

En una cadena binaria, si cada locus muta de forma independiente con intensidad $G = 0,5$, cada cadena binaria tiene la misma probabilidad de aparecer. Para una cadena de longitud n , cualquier cadena específica tiene probabilidad

$$\left(\frac{1}{2}\right)^n.$$

Por eso, desde cualquier individuo binario se puede llegar a cualquier otro individuo del espacio de búsqueda con probabilidad positiva y uniforme.

Algunos ejemplos discretos son:

numValues	$G = 1 - 1/\text{numValues}$
2	0,5
4	0,75
10	0,9
100	0,99

En variables reales, G debe definirse en las unidades del gen. Si una variable x vive en $[a, b]$, entonces G debe ser comparable con el ancho $b - a$, o con una fracción suficientemente grande de ese ancho, según el operador de mutación usado. Por ejemplo, si $x \in [0, 1]$, una escala global podría ser 0,5 o 1,0; si $x \in [-10, 10]$, podría ser 10 o 20. La elección exacta depende de si la mutación suma ruido uniforme, ruido gaussiano u otro tipo de perturbación.

L , en cambio, debe interpretarse como la menor intensidad local que todavía produce un cambio significativo para el resultado. No debe elegirse sólo como un número pequeño, sino como una escala de precisión. Si L es demasiado grande, el algoritmo no puede afinar soluciones; si L es demasiado pequeño, muchas mutaciones serán irrelevantes y se desperdiciarán evaluaciones.

En cadenas binarias de longitud n , una elección local natural es

$$L \approx \frac{1}{n},$$

porque produce aproximadamente un bit cambiado por mutación. Por ejemplo:

n	$L \approx 1/n$
10	0,1
50	0,02
100	0,01
1000	0,001

En cadenas de enteros, si lo mínimo significativo es cambiar un solo locus, también puede usarse $L \approx 1/n$, donde n es la longitud del genoma. Si los enteros son ordinales y el tamaño del paso importa, L debe reflejar el menor paso significativo. Por ejemplo, en valores $0, \dots, 99$, si una diferencia de una unidad importa, L puede asociarse con un paso de $1/99$ en escala normalizada; si sólo importan cambios de cinco unidades, entonces la escala local relevante es $5/99$.

En variables reales, L debe ser la precisión mínima útil. Si $x \in [0, 1]$ y diferencias menores que 0,001 no son relevantes, entonces $L = 0,001$ es una escala local razonable. Si $x \in [0, 1000]$ y diferencias menores que 5 unidades no cambian la evaluación de forma significativa, entonces $L = 5$ describe mejor la resolución necesaria.

En resumen, G responde: “¿puedo alcanzar cualquier región del espacio de búsqueda?”. L responde: “¿cuál es el menor cambio que todavía vale la pena evaluar?”.

El supuesto matemático deseado es $G > L$, porque entonces el exponente de mutación resulta positivo y la mutación crece conforme aumenta el rango normalizado r_i , es decir, conforme se avanza desde el mejor individuo hacia el peor en la población ordenada por fitness.

5. Representación de individuos y genes

5.1. Estructura de un individuo

Cada individuo contiene:

- un arreglo ordenado de genes;
- un fitness cacheado;
- un rango entero;
- la última intensidad de mutación aplicada;
- el rango de la pareja usada en la última recombinación;
- un campo textual auxiliar para logging.

La copia de un individuo es **profunda** a nivel de genes: el constructor de copia llama, para cada locus, a `problem.getNewGene(gene)`.

5.2. Importancia del contrato de copia

Este detalle es más importante de lo que parece. Si la copia de genes no se hace correctamente, la independencia estocástica entre individuos puede romperse y la interpretación experimental del algoritmo deja de ser fiable.

La lección metodológica es clara:

- el operador de copia no es un detalle administrativo;
- forma parte esencial de la semántica del algoritmo;
- una copia incorrecta puede falsear conclusiones experimentales.

5.3. Dos ejemplos de genes del repositorio

GeneInteger. Representa un gen discreto en $\{0, 1, \dots, \text{NUM_VALUES} - 1\}$. Su mutación interpreta la intensidad como probabilidad de reemplazo por otro valor distinto del actual. Es decir, en este caso la intensidad sí termina siendo usada como probabilidad, pero esa decisión se toma a nivel del gen, no del motor.

GeneDoublePrecision. Representa un gen real. Su mutación aplica ruido gaussiano aditivo con desviación estándar igual a la intensidad recibida.

6. Ciclo evolutivo completo

6.1. Esquema general

Para un problema dado, una semilla, un tamaño de población N y un número de repeticiones R , el ciclo externo es:

1. crear una población inicial aleatoria;
2. evaluarla y ordenarla;
3. registrar un snapshot inicial;
4. repetir mientras no se alcance el fitness objetivo y no se haya agotado la paciencia:
 - a) seleccionar;
 - b) recombinar;

- c)* reevaluar;
 - d)* registrar mejora si hubo;
 - e)* mutar;
 - f)* reevaluar;
 - g)* registrar mejora si hubo;
 - h)* adaptar el problema si implementa `AdaptiveProblem`.
5. registrar un snapshot final;
 6. añadir una fila al `summary.csv`;
 7. al final de todas las repeticiones, generar gráficas automáticamente.

6.2. Inicialización

La población inicial se crea con:

- tamaño fijo N ;
- individuos aleatorizados;
- un `Random` derivado de la semilla base de la corrida y del índice de repetición.

Después se evalúa a toda la población y se ordena de fitness mayor a menor. El número inicial de evaluaciones de fitness es exactamente N .

7. Selección por rango

7.1. Idea

La selección no usa directamente el valor crudo del fitness. En su lugar:

1. la población ya está ordenada de mejor a peor;
2. cada individuo recibe un rango implícito por su posición;
3. ese rango determina cuántos clones se espera producir.

7.2. Presión selectiva fija

La implementación actual fija la presión selectiva en

$$S = 3.$$

No es un parámetro configurable; es una decisión de diseño del algoritmo.

7.3. Definición del rango

En la selección, el código usa

$$r_i = \frac{i}{N-1},$$

donde:

- $i = 0$ corresponde al mejor individuo;
- $i = N - 1$ corresponde al peor;
- por lo tanto $r_i \in [0, 1]$.

En este documento, cuando se diga que un individuo está “arriba” de la población ordenada, significa que tiene mejor fitness y un rango normalizado cercano a 0. Cuando se diga que está “abajo”, significa que tiene peor fitness y un rango normalizado cercano a 1. Por tanto, un valor mayor de r_i no significa “mejor rango”, sino peor posición relativa dentro de la población ordenada.

En la implementación que este documento describe, la definición correcta es $r_i = i/(N - 1)$.

7.4. Conteo esperado de clones

El número esperado de clones del individuo i es

$$c_i = S(1 - r_i)^{S-1}.$$

Como $S = 3$, se vuelve

$$c_i = 3(1 - r_i)^2.$$

7.5. Materialización en el código

La implementación hace dos pasos:

1. asignación determinista:

$$n_i = \lfloor c_i \rfloor;$$

2. redondeo estocástico sobre la parte fraccional hasta completar exactamente N individuos.

Es decir, si

$$f_i = c_i - \lfloor c_i \rfloor,$$

entonces se hacen pasadas cíclicas y se agrega un clon extra al individuo i con probabilidad f_i , mientras el total de clones siga siendo menor que N .

7.6. Observaciones técnicas

- La selección conserva exactamente el tamaño de población.
- El procedimiento usa redondeo estocástico secuencial porque eso basta para materializar la ley de clonación adoptada por el diseño.
- El redondeo estocástico introduce un pequeño sesgo de orden por el recorrido secuencial, pero actualmente ese sesgo se acepta como parte del diseño.

8. Emparejamiento por rango y recombinación

8.1. Por qué el emparejamiento por rango es central

En este framework, el rango no solo decide cuántos clones sobreviven y qué intensidad de mutación recibe cada individuo. También decide la **vecindad reproductiva**.

Eso significa que el rango organiza simultáneamente tres mecanismos:

1. la presión selectiva;
2. la intensidad de mutación;
3. y la estructura de apareamiento.

Por eso el emparejamiento por rango debe considerarse una característica central del algoritmo y no un detalle de implementación. Si el emparejamiento fuera aleatorio, la semántica del método cambiaría de manera importante, porque se perdería la relación sistemática entre orden por fitness y mezcla genética.

8.2. Emparejamiento

La recombinación se hace en parejas adyacentes:

$$(0, 1), (2, 3), (4, 5), \dots$$

No hay emparejamiento aleatorio global. El criterio es puramente posicional después del ordenamiento por fitness.

8.3. Operador

El operador de recombinación a nivel de individuo es cruzamiento uniforme con **intercambio in situ**. Este detalle es esencial:

- la selección ya produjo una nueva población compuesta por clones;
- la recombinación no construye objetos “hijo” adicionales;
- en su lugar, toma dos clones ya existentes y les intercambia genes locus por locus.

Por tanto, la secuencia correcta del algoritmo es:

1. la selección multiplica genotipos mediante clonación;
2. la población clonada reemplaza a la anterior;
3. la recombinación actúa sobre esa población ya clonada;
4. cada pareja de clones se modifica directamente por intercambio de loci.

No hay una fase separada de “generación de descendencia” en el sentido clásico de crear dos hijos nuevos a partir de dos padres intactos. Los “padres” que entran a recombinación son exactamente los individuos clonados que sobreviven a la selección, y después del intercambio esos mismos objetos quedan convertidos en las soluciones recombinadas que siguen al resto del ciclo.

En cada pareja, el mecanismo es:

- para cada locus j ,
- con probabilidad 0,5,
- se intercambian los genes de ambos individuos en ese locus.

Formalmente, si x_j y y_j son los genes de dos padres, entonces:

$$(x_j, y_j) \leftarrow \begin{cases} (y_j, x_j), & \text{con probabilidad 0,5,} \\ (x_j, y_j), & \text{con probabilidad 0,5.} \end{cases}$$

8.4. Consecuencias

- Los individuos cercanos en rango tienden a recombinarse entre sí.
- La mezcla genética está sesgada por la estructura del ordenamiento.
- La recombinación opera sobre clones seleccionados, no sobre una población parental separada de una población filial.
- El operador no aumenta el tamaño de la población, porque no crea individuos nuevos; solo transforma los ya existentes.
- El operador es simétrico.
- No privilegia ni prefijos ni sufijos del genoma.
- Conserva la distribución marginal de genes, aunque mezcla bloques de construcción a nivel de loci.

9. Mutación por rango

9.1. Idea central

La mutación no es plana. La intención del diseño es:

- preservar más a los individuos ubicados arriba de la población ordenada, es decir, con mejor fitness y r_i cercano a 0;
- perturbar más a los individuos ubicados abajo, es decir, con peor fitness y r_i cercano a 1;
- hacer coexistir explotación cerca del frente y exploración en la cola de la población.

9.2. Exponente de mutación

El framework define

$$\beta = \frac{\ln(G/L)}{\ln(N-1)}.$$

Si $G > L$, entonces $\beta > 0$.

9.3. Intensidad por individuo

Para el individuo de rango i , la intensidad es

$$I_i = G r_i^\beta,$$

con

$$r_i = \frac{i}{N-1}.$$

De aquí se sigue que:

- para el mejor individuo, $r_0 = 0$ y por tanto $I_0 = 0$;
- para el segundo mejor individuo, $r_1 = \frac{1}{N-1}$, y entonces

$$I_1 = G \left(\frac{1}{N-1} \right)^\beta = L;$$

- para el peor individuo, $r_{N-1} = 1$ y por tanto $I_{N-1} = G$;
- la intensidad crece monótonamente con r_i si $\beta > 0$.

Ese detalle no es accidental. La forma en que se define β hace que el perfil de intensidades quede anclado exactamente en tres puntos:

$$I_0 = 0, \quad I_1 = L, \quad I_{N-1} = G.$$

Por tanto, L no representa la intensidad del mejor individuo, sino la del primer individuo no elitista, es decir, del segundo mejor elemento de la población ordenada.

9.4. Interpretación correcta

Aquí conviene ser muy preciso. RankGA no define por sí mismo qué significa I_i . Solo define *cuánto* recibe cada individuo en una escala abstracta. El significado operacional depende del gen:

- el mejor individuo tiene intensidad exactamente 0;
- el segundo mejor individuo tiene intensidad exactamente L ;
- el peor individuo tiene intensidad exactamente G .

Ese anclaje no es una interpretación opcional, sino una consecuencia directa de la forma en que el algoritmo define r_i y β .

- en genes reales, I_i puede ser desviación estándar del ruido;
- en genes categóricos, puede usarse como probabilidad de reemplazo;
- en otras codificaciones podría representar radio de vecindad o magnitud de perturbación.

Este desacoplamiento hace al framework flexible, pero también obliga a que cada problema mantenga un contrato semántico consistente.

10. Evaluación y detección de mejora

10.1. Momento de evaluación

Dentro de una iteración del ciclo, el fitness se recalcula dos veces:

1. después de recombinación;
2. después de mutación.

La selección no evalúa porque trabaja con copias de individuos cuyo fitness ya había sido conocido antes del clonaje.

10.2. Criterio de mejora

El framework actual ya no usa un único criterio de “mejora” para todo. Hay que distinguir entre:

- mejora estricta de fitness;
- reemplazo del incumbente;
- y reinicio de paciencia.

En todos los casos se exige además que el nuevo candidato no sea genotípicamente idéntico al incumbente previo. En código, la distancia cuadrática entre ambos debe ser positiva.

Modo strict. Si la política de incumbente es **strict**, entonces el reemplazo del incumbente exige:

- el fitness debe ser estrictamente mayor;
- la distancia cuadrática entre genomas debe ser positiva.

Modo neutral. Si la política de incumbente es **neutral**, entonces también se permite sustituir al incumbente cuando:

- el fitness es igual al del incumbente actual;
- pero la distancia cuadrática entre genomas sigue siendo positiva.

Es decir, en modo **neutral** sí se permite que el representante del mejor fitness alcanzado se desplace sobre una meseta neutral. Lo que no se permite en ningún caso es reemplazarlo por el mismo genotipo.

10.3. Distancia entre individuos

La distancia entre individuos se calcula como

$$d^2(x, y) = \sum_{j=1}^n (d_j(x_j, y_j))^2,$$

donde d_j es la distancia definida por el gen en el locus j .

En genes enteros tipo Hamming unilocus, cada término vale 0 o 1. En genes reales, depende de la implementación concreta del gen.

11. Adaptación opcional del problema

Si un problema implementa `AdaptiveProblem`, entonces el driver llama:

`adapt(bestFitness)`

después de completar la iteración evolutiva.

Esto permite que el problema:

- ajuste pesos internos;
- cambie parámetros de representación;
- modifique penalizaciones;
- o altere alguna heurística dependiente del progreso.

Si el problema no implementa esa interfaz, no ocurre nada. La adaptación no forma parte del contrato básico de `Problem`.

12. Criterios de paro

12.1. Paro por objetivo

La corrida termina si el mejor fitness actual satisface

$$f_{\text{máx}} \geq f_{\text{goal}},$$

donde $f_{\text{goal}} = \text{problem.getGoalFt}()$.

12.2. Paro por paciencia

También termina si transcurre un tiempo fijo sin el tipo de progreso que la política de paciencia considere suficiente.

La versión actual permite parametrizar dos cosas distintas:

- el criterio de reemplazo del incumbente;
- y el criterio de reinicio de paciencia.

La paciencia se configura con un parámetro en milisegundos:

`patienceMillis`.

Por defecto:

`patienceMillis = 60 000 ms = 1 minuto`.

12.3. Política de incumbente

El framework distingue dos modos:

- **strict**: el incumbente solo se sustituye si aparece un individuo con fitness estrictamente mayor;
- **neutral**: el incumbente también puede sustituirse por otro genotipo con el mismo fitness, siempre que la distancia genotípica sea positiva.

Esto controla si el representante del mejor fitness alcanzado puede o no desplazarse sobre una meseta neutral.

12.4. Política de reinicio de paciencia

El reinicio de paciencia también es configurable:

- **fitness**: la paciencia se reinicia solo con mejora estricta de fitness;
- **movement**: la paciencia se reinicia cuando cambia el incumbente según la política de incumbente seleccionada.

La distinción es importante. Permitir movimiento neutral del incumbente no obliga a interpretar ese movimiento como progreso suficiente para seguir extendiendo la corrida. Son dos decisiones metodológicas distintas.

12.5. Consecuencia importante

La paciencia es **por repetición**, no por experimento completo. Si se ejecutan 100 repeticiones y ninguna alcanza el objetivo, el tiempo total puede escalar aproximadamente a 100 minutos, salvo pequeñas variaciones por inicialización, I/O y sistema operativo.

13. Contabilidad de evaluaciones

13.1. Qué se cuenta

La variable `evaluations` cuenta cuántas veces se evaluó fitness en una repetición. Se suma:

- N al inicializar;
- N después de recombinación en cada generación;
- N después de mutación en cada generación.

13.2. Corrección importante ya incorporada

La versión actual ya no fuerza una generación extra si la población inicial alcanza el objetivo. El ciclo principal usa un `while` y no un `do-while`. Eso evita sobrecontar evaluaciones en casos donde la solución objetivo ya aparece desde el principio.

14. Salida experimental

14.1. Logs heredados

Por cada corrida se generan:

- un archivo `.csv` con snapshots cronológicos del mejor individuo;
- un archivo `_rep.csv` por repetición con el último estado de la población en esa repetición.

14.2. Resumen estructurado

También se genera un `summary.csv` con una fila por repetición. Sus columnas son:

- índice de repetición;
- semilla efectiva de la repetición;
- número de evaluaciones;
- mejor fitness final;
- tiempo transcurrido en milisegundos;
- razón de terminación.

La información que no cambia por repetición se escribe en el archivo de metadata asociado, `_summary_meta.csv`. Ahí quedan, entre otros:

- algoritmo;
- clase del problema;
- nombre del problema;
- parámetros efectivos del problema;
- semilla base;

- `run_id`;
- tamaño de población;
- longitud del genoma;
- número total de repeticiones;
- fitness objetivo;
- paciencia configurada en milisegundos;
- política de reemplazo del incumbente;
- política de reinicio de paciencia;
- prefijo del archivo de salida.

14.3. Gráficas automáticas

Al terminar todas las repeticiones, RankGA invoca un script en Python que genera:

- una gráfica ordenada por evaluaciones;
- una gráfica ordenada por tiempo transcurrido;
- y dos CSV ordenados correspondientes.

La versión actual ya incorpora el `runId` en el nombre de la figura para evitar colisiones entre corridas con el mismo problema y la misma semilla.

15. Ejemplo: OneMax de 8 bits

15.1. Definición

En el problema OneMax de longitud $n = 8$:

- el genoma es binario;
- el fitness es la suma de bits en uno;
- el objetivo es alcanzar fitness 8.

Formalmente, si $x \in \{0, 1\}^8$, entonces

$$f(x) = \sum_{j=1}^8 x_j.$$

15.2. Configuración en el repositorio

La implementación actual:

- usa `GeneInteger` con dominio binario;
- define intensidad local $L = 1/8$;
- define intensidad global $G = 0,5$;
- usa una estrategia de copia de genes que preserva la independencia estocástica entre individuos.

15.3. Interpretación experimental

OneMax es un benchmark deliberadamente sencillo. Si este problema falla con frecuencia por paciencia, eso suele indicar:

- un defecto en la implementación;
- una semántica inconsistente de mutación;
- o un error en el diseño experimental y no una limitación genuina del algoritmo.

16. Supuestos y limitaciones del framework

16.1. Supuestos fuertes

El algoritmo presupone:

- fitness orientado a maximización;
- poblaciones de tamaño al menos 3;
- $G > L$ para que la intensidad de mutación crezca de forma monótona con r_i ;
- una implementación consistente del operador de copia de genes;
- y una interpretación bien definida de intensidad dentro de cada tipo de gen.

16.2. Decisiones de diseño

- La presión selectiva S es fija.
- El emparejamiento para recombinación es adyacente y no aleatorio.
- La selección usa redondeo estocástico secuencial y no necesita muestreo multinomial exacto para cumplir su objetivo.

16.3. Limitaciones reales de implementación

- El estado mutable de ejecución pertenece a una instancia de **RankGA**. Las llamadas públicas de conveniencia siguen creando un *runner* simple por corrida, pero el estado ya no queda compartido como estado estático global.
- La generación automática de gráficas depende de componentes externos disponibles en el entorno.

16.4. Criterio documental

Para entender el comportamiento real del framework, hay que preferir:

1. el código ejecutable;
2. la ejecución efectiva de los tests automáticos del proyecto, cuando son aplicables al cambio o a la afirmación que se quiere sostener;
3. las corridas reproducibles y sus resúmenes experimentales estructurados;
4. y la revisión manual de archivos de salida cuando hace falta una validación complementaria.

16.5. Observación práctica sobre verificación

En este repositorio conviene distinguir entre:

- compilación correcta del código;
- ejecución efectiva de pruebas automáticas;
- y validación manual mediante corridas, inspección de logs, `summary.csv` y gráficas.

Desde el punto de vista metodológico, esa distinción importa porque una afirmación del tipo “esto ya está probado” no significa lo mismo si se basa en una batería automatizada completa, en una corrida reproducible o en una inspección manual del *summary*. En una descripción rigurosa del framework, esas fuentes de evidencia no deben confundirse ni presentarse como si tuvieran exactamente el mismo alcance.

17. Resumen conceptual final

RankGA puede resumirse como sigue:

1. ordena la población por fitness;
2. replica individuos según su rango, no según el valor crudo del fitness;
3. cruza por parejas adyacentes;
4. muta con intensidad creciente conforme aumenta r_i , es decir, conforme se avanza hacia individuos de peor fitness;
5. preserva al mejor individuo porque su intensidad es cero;
6. asigna al segundo mejor individuo exactamente la intensidad local L ;
7. permite adaptación opcional del problema;
8. y produce salidas estructuradas para análisis experimental.

La contribución práctica del framework no es solo combinar selección por rango con mutación dependiente del rango, sino hacerlo de manera genérica para varias codificaciones y problemas. El precio de esa generalidad es que el contrato entre motor y genes debe estar especialmente bien cuidado. Cuando ese contrato se respeta, el comportamiento es razonable y reproducible; cuando se viola, incluso problemas muy fáciles pueden dar resultados engañosos.

18. Explicación intuitiva de la dinámica por rango

Esta sección no introduce nuevas ecuaciones ni sustituye la descripción formal anterior. Su objetivo es ofrecer una **lectura operativa** de la dinámica del algoritmo: cómo se reparte funcionalmente la población y por qué el diseño combina protección de la élite con exploración persistente. Lo que sigue debe entenderse como explicación del *comportamiento esperado de la implementación actual*, no como una demostración matemática general de todas sus trayectorias posibles.

18.1. La población no cumple una sola función

Este algoritmo está diseñado para que la población no reciba un tratamiento uniforme. No todos los individuos cumplen la misma función. La población se ordena por fitness y a cada posición se le asocia un rango normalizado r_i . Ese rango determina el papel de cada individuo dentro de la búsqueda.

La intuición general es:

- en la parte alta de la población ordenada, donde r_i es cercano a 0, se protege y se explota lo mejor encontrado;
- al avanzar hacia posiciones con r_i mayor, la búsqueda se vuelve cada vez más amplia;
- en la parte baja de la población ordenada, donde r_i es cercano a 1, se mantiene exploración fuerte y permanente.

Así, el rango normalizado no solo resume la posición por calidad. También organiza qué tipo de búsqueda hace cada zona de la población.

18.2. Selección por clonación según rango

Antes de recombinar, el algoritmo aplica una selección por clonación. Los individuos mejores reciben más copias y los peores menos. Esto introduce presión selectiva, pero de manera graduable y no abrupta.

La idea no es simplemente “eliminar a los peores”, sino dar más presencia reproductiva a los mejores sin destruir de golpe la diversidad de la población.

Aquí aparece un caso especialmente importante: $S = 3$. Con $S = 3$, el mejor individuo tiene conteo esperado de clones igual a 3, porque para la mejor posición se cumple $r_0 = 0$ y por tanto:

$$c_0 = 3(1 - r_0)^2 = 3.$$

Esa observación es importante porque hace plausible que el mejor quede representado varias veces en la parte alta de la población antes de recombinar. En la implementación actual, esa multiplicidad no es una regla de elitismo externa: emerge de la ley de clonación por rango.

18.3. Qué implica $S=3$ para la parte alta

Cuando el mejor tiene tres copias y la población se vuelve a ordenar por fitness, esas copias tienden a quedar al inicio, donde r_i es cercano a 0.

Luego viene la recombinación por parejas adyacentes:

- los dos primeros se recombinan entre sí;
- si son copias idénticas del mejor, la recombinación uniforme no cambia nada;
- por tanto, esos dos primeros quedan sin cambios.

Eso significa que el mejor queda protegido sin necesidad de meter un elitismo forzado como regla externa. La propia dinámica del algoritmo preserva una señal del mejor.

Luego, el tercer clon del mejor se recombina con el que era el siguiente individuo en esa zona alta de la población ordenada. En situaciones típicas, eso coincide con un individuo muy bueno, a menudo el segundo mejor previo a la selección. La lectura funcional es razonable: una parte del mejor se preserva intacta y otra parte se usa para explotar localmente la región donde ya viven las mejores soluciones.

18.4. Por qué recombinar buenos con buenos tiene sentido

Si el mejor y el segundo mejor son buenos por razones parcialmente distintas, puede ocurrir que:

- compartan algunos genes valiosos;
- cada uno tenga además otros genes buenos que el otro no tiene.

En ese caso, al recombinarlos puede suceder algo interesante:

- los genes buenos comunes se conservan;
- y los genes buenos distintos pueden intercambiarse.

Así puede aparecer un descendiente que reúna partes buenas de ambos.

Por eso no se trata solo de “proteger al mejor”. También se trata de usar parte de él para explotar con mayor resolución la región donde ya se encuentran soluciones fuertes.

18.5. Por qué la recombinación uniforme ayuda

La recombinación uniforme no supone que los genes valiosos estén juntos en el cromosoma. En cambio, lo que hace es proteger los **alelos comunes** entre los dos padres, estén contiguos o no.

Eso importa porque, si dos individuos buenos comparten genes valiosos, esos genes comunes no se rompen. La recombinación solo actúa donde los padres difieren.

Entonces, cuando se recombinan individuos de posición parecida, sobre todo individuos buenos, lo normal es que:

- lo que ambos ya tienen bien quede preservado;
- y la mezcla ocurra justo en las partes donde todavía hay diferencias.

Esa es una razón importante por la que la recombinación entre individuos buenos resulta una estrategia coherente dentro del diseño.

18.6. Emparejamiento por rango

Los individuos no se recombinan al azar. Se recombinan con otros de posición parecida en la población ordenada.

Esto también está diseñado a propósito:

- los buenos se recombinan con buenos;
- los malos con malos;
- los intermedios con intermedios.

La razón es clara: si un individuo muy malo o muy exploratorio se recombinara con un individuo muy bueno, tendería a destruir parte del material genético valioso del bueno.

En cambio:

- buenos con buenos favorece explotación y refinamiento;
- malos con malos favorece exploración amplia.

Así, el emparejamiento por rango protege a la élite y, al mismo tiempo, deja a la parte baja de la población funcionando como zona de exploración.

18.7. Qué pasa después: mutación según rango

Después de recombinar, se aplica mutación, pero la intensidad no es la misma para todos. La regla se construye para que:

- el mejor reciba intensidad 0;
- el segundo reciba intensidad L ;
- el peor reciba intensidad G ;
- y el resto reciba intensidades intermedias de manera continua.

Eso fija tres comportamientos clave:

- el mejor se conserva;
- el segundo hace búsqueda local;
- el peor hace búsqueda global o muy amplia, según la semántica del gen concreto.

18.8. Qué significa que el segundo reciba L

Aquí vuelve a conectar el caso $S = 3$.

Después de recombinar:

- los dos primeros no cambiaron, porque eran copias idénticas;
- si la recombinación del tercer clon con otro individuo muy bueno todavía no produjo algo superior;
- entonces el segundo mejor sigue siendo una copia del mejor.

Y a ese segundo se le asigna exactamente L .

Eso significa que una copia del mejor pasa a hacer búsqueda local alrededor del mejor actual.

Es una parte especialmente elegante del algoritmo:

- una copia del mejor queda intacta;
- otra copia del mejor queda arriba en la población ordenada;
- y esa copia recibe una mutación pequeña que explora localmente.

Por tanto, no solo se protege al mejor: también se explora de manera fina alrededor de él.

18.9. Qué ocurre al avanzar hacia rangos normalizados mayores

La dinámica cambia gradualmente al avanzar desde r_i cercano a 0 hacia r_i cercano a 1. Dicho de otra forma: cambia al pasar de los individuos de mejor fitness a los de peor fitness.

En la parte alta de la población ordenada:

- los individuos se parecen mucho entre sí;
- la recombinación cambia poco;
- la mutación es pequeña o nula.

Ahí la búsqueda es local y fina.

Más abajo:

- los individuos que se recombinan entre sí son cada vez más distintos;
- la recombinación produce descendientes potencialmente más alejados de sus padres;
- la mutación también es cada vez mayor.

Por tanto, al avanzar hacia r_i mayores, la búsqueda se va haciendo cada vez más amplia.

Eso significa que el algoritmo no separa bruscamente “explotación” y “exploración”. Lo que hace es una **transición gradual** desde una a la otra.

18.10. Qué papel tienen los peores

Los peores individuos no están ahí solo como residuo. Cumplen una función muy importante.

Como se recombinan entre sí y además mutan con intensidad alta, generan individuos lejanos y exploratorios.

Eso mantiene diversidad genética de forma permanente mientras la corrida siga activa.

Entonces, la parte baja de la población ordenada funciona como una zona de exploración amplia. Allí aparecen nuevas combinaciones y nuevas mutaciones que pueden alejarse bastante de lo ya consolidado en la élite.

18.11. Qué pasa si un individuo exploratorio resulta bueno

Puede ocurrir que un individuo proveniente de esa zona exploratoria:

- haya surgido de recombinación entre malos;
- haya recibido mutación fuerte;
- y aun así resulte ser muy bueno.

Cuando eso ocurre, no hace falta introducir una regla especial para “rescatarlo”. Basta con volver a ordenar por fitness.

Su nueva posición y su nuevo r_i determinan con quién se recombina a continuación. Si ahora es bueno, pasará a recombinarse con otros buenos.

Eso es importante porque un individuo lejano puede resultar bueno por varias razones:

- porque contiene los mismos genes buenos que otros buenos;
- porque contiene genes buenos distintos;
- o por una combinación de ambas cosas.

Sea cual sea el caso, si sube hacia la parte alta de la población ordenada, el algoritmo empieza a tratarlo como un individuo bueno. Esa es una propiedad fuerte del diseño.

18.12. Por qué los exploratorios no deben recombinarse con los mejores mientras sigan siendo malos

Si un individuo muy mutado o muy lejano sobrevive, pero sigue estando entre los peores que sobrevivieron, entonces debe seguir recombinándose con los peores.

No debe recombinarse con los mejores porque, en ese caso, lo normal es que destruya parte de los buenos genes del bueno.

Por eso el emparejamiento por rango no es un detalle accesorio. Es una parte central de la protección del material genético valioso.

18.13. Qué tipo de búsqueda implementa el algoritmo

Visto globalmente, el algoritmo organiza la población en franjas funcionales.

Arriba de la población ordenada:

- preservación;
- búsqueda local;
- recombinación entre individuos muy buenos.

En la zona media:

- recombinación entre individuos todavía buenos, pero más distintos;
- mutación moderada;
- exploración intermedia.

Abajo de la población ordenada:

- recombinación entre individuos muy distintos;
- mutación fuerte;
- exploración amplia o global.

Entonces la población completa cubre distintas escalas de búsqueda al mismo tiempo.

18.14. Sobre convergencia y diversidad

Aquí conviene usar una definición precisa. Si por convergencia se entiende pérdida irreversible de diversidad genética tal que el algoritmo ya no pueda volver a generar variantes lejanas, entonces la dinámica actual no está diseñada para empujar fuertemente a ese escenario, porque la parte baja de la población ordenada sigue introduciendo diversidad de manera persistente:

- por recombinación entre individuos lejanos;
- y por mutación fuerte.

Eso **soporta** la afirmación operativa de que el algoritmo mantiene una fuente permanente de exploración amplia. Lo que **no** debe afirmarse sin prueba adicional es una imposibilidad formal de convergencia para toda instancia, toda codificación y toda trayectoria estocástica.

18.15. La persistencia de la élite en una región no es, por sí sola, un problema

Puede ocurrir que los mejores individuos permanezcan durante mucho tiempo en una misma región del espacio de búsqueda.

Eso no equivale, por sí solo, a convergencia patológica. Solo significa que, por el momento, esa sigue siendo la mejor región encontrada.

Mientras la parte baja siga generando diversidad y exploración amplia, sigue existiendo la posibilidad de descubrir otra región mejor. Si eso ocurre, esa novedad puede subir hacia la parte alta de la población ordenada y desplazar a la élite actual.

18.16. Resumen intuitivo final

La dinámica completa del algoritmo puede entenderse así:

- la selección por clonación da más representación a los mejores;
- con $S = 3$, el mejor queda automáticamente protegido sin elitismo forzado;
- la recombinación por rango hace que los mejores cooperen con mejores y los peores con peores;
- la recombinación uniforme protege los alelos comunes entre padres;
- la mutación según rango hace que arriba haya búsqueda local y abajo búsqueda amplia;
- si aparece un individuo exploratorio bueno, su nueva posición lo reposiciona automáticamente;
- la parte baja de la población mantiene diversidad genética de forma persistente.

En pocas palabras: el algoritmo está diseñado para que la parte alta de la población refine lo mejor encontrado, mientras la parte baja sigue buscando lejos, y ambas zonas queden conectadas por una transición gradual gobernada por el rango normalizado.